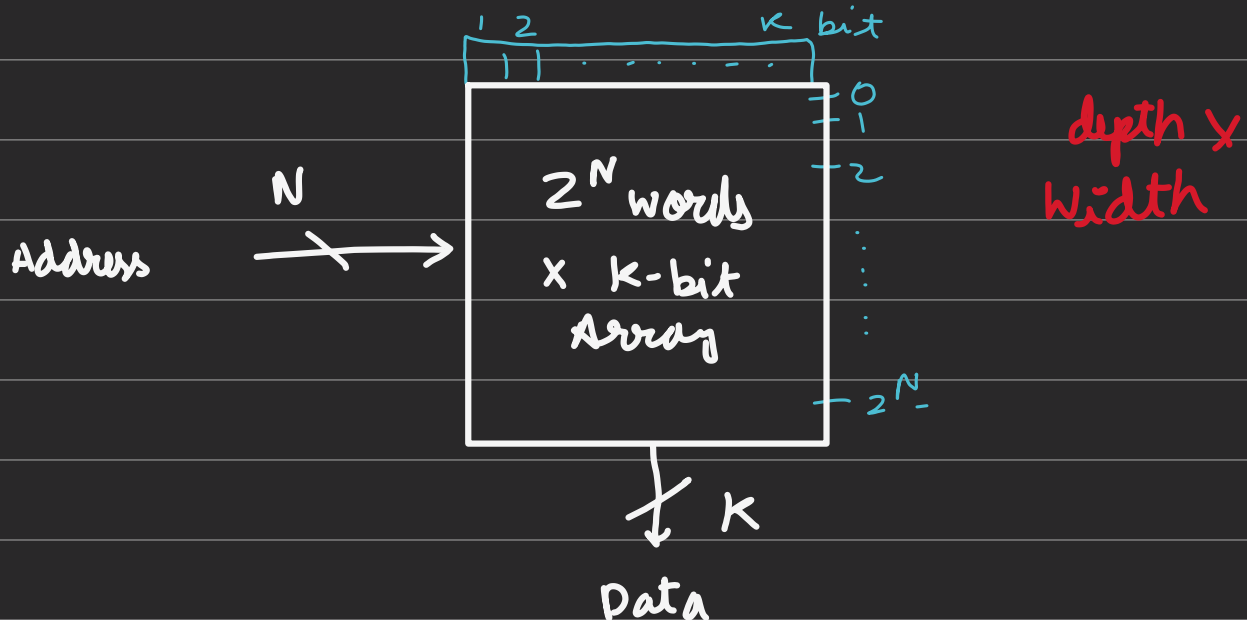
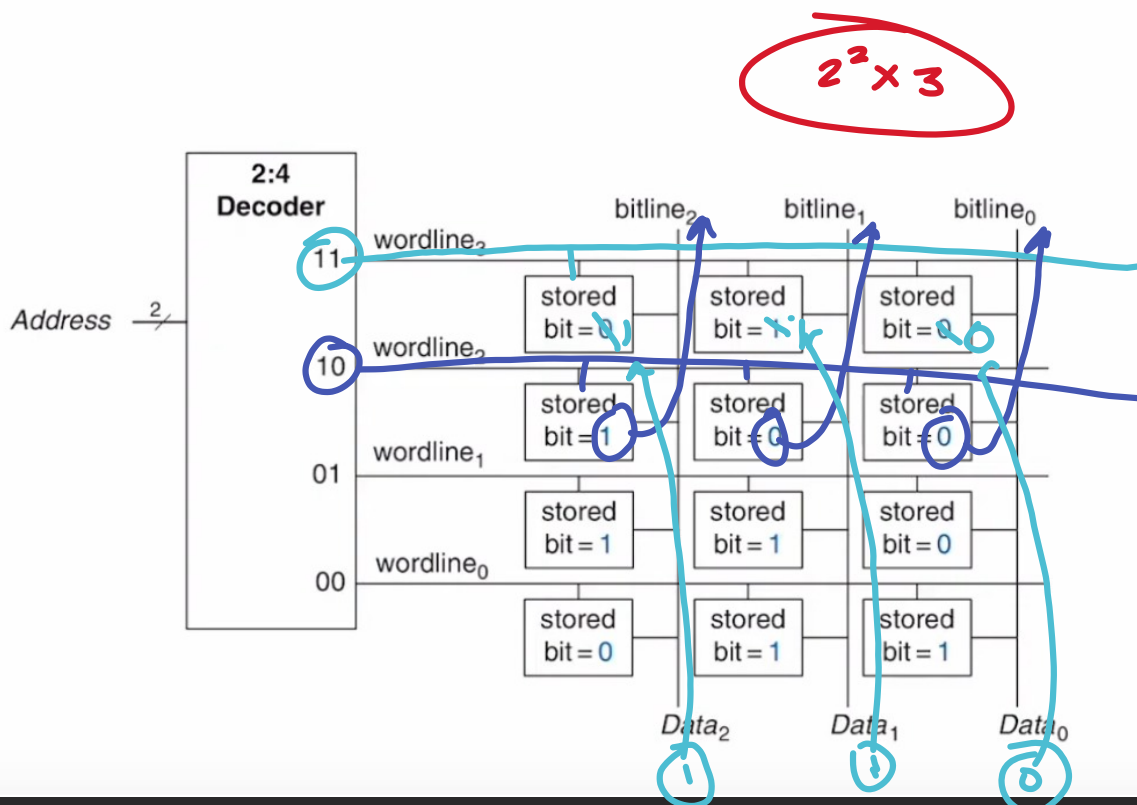


Memory Arrays

i.e

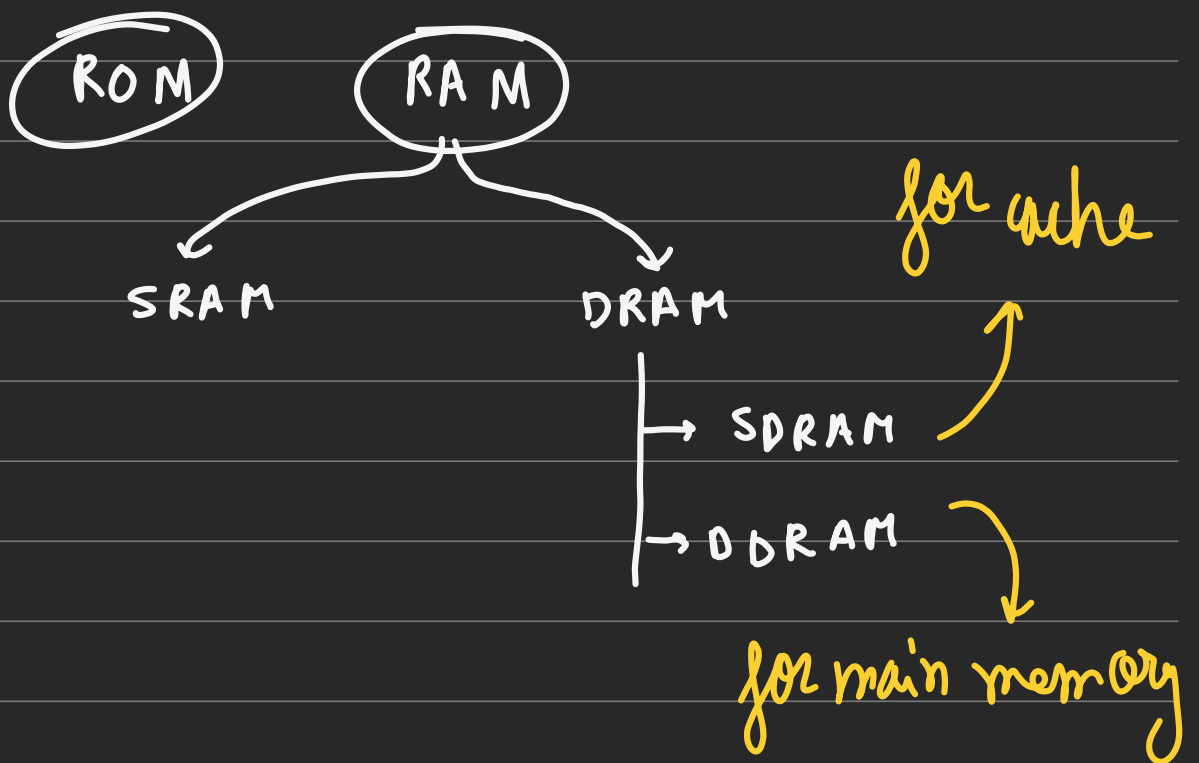


Bit Cells Organization



- active wordline, by giving address & output will come on [2:0] bitline
- If want write on particular address put [2:0] Data, then put address.

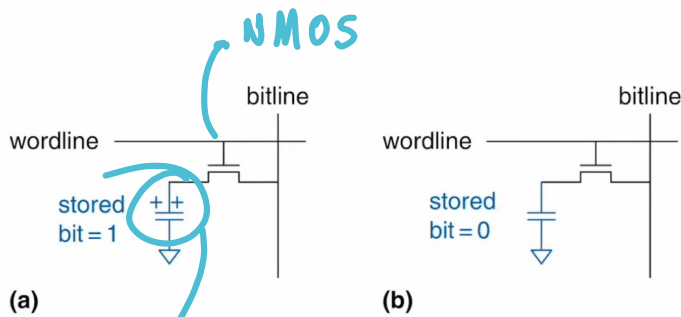
Memory Types



* RAM : all address can be accessed in equal time, because they are accessed randomly.

↳ volatile

Dynamic Random Access Memory (DRAM)



capacitor get charged

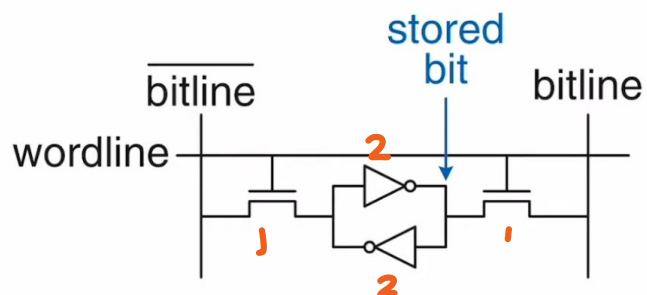
DRAM	SRAM
cheap.	expensive
high density	low density.
slow	much faster
→ for large storage needs.	

- after reading cap. discharges, so need extra circuitry to refresh/charge then continuously.
- so it's overall slow

• we require 6 transistor to make one bit cell

• so consume more space

Static Random Access Memory (SRAM)



Type	Transistor	Speed
S.S.	~ 20	fastest
SRAM	6	Medium
DRAM	1	slow

→ Low power consumption, small area

so DRAM are further researched & developed into SDRAM & DDRAM

Laptop
DDR4

(Double data Dynamic RAM)

they work on rising edge & falling edge of clock

Read Only Memory (ROM)

(Programmable ROM)





designed by manufacturer

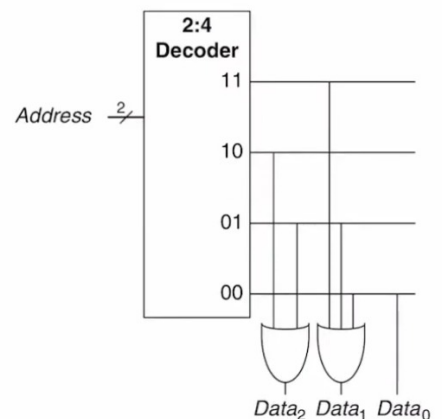
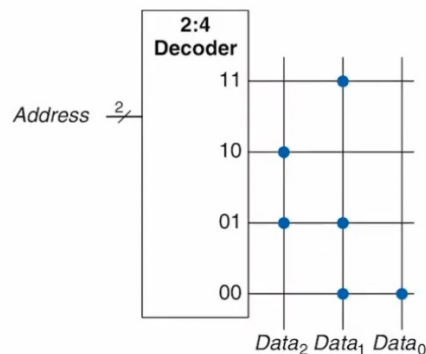
fuses were made from photo resistors, so when kept in sunlight/UV they join again

then came EPROM (Erasable PROM)

EEPROM (electrical EPROM)
i.e. flash

using OR gates

Read Only Memory (ROM)



HDL for Memory Arrays

2-Port

(1 async Read / 1 Sync Write)



module rom 2 port

(parameter Addr.width = 3, Data.width = 8)

(
——
——

input [Addr.width-1:0] `r_addr`

—— `w_addr`

[Data.width-1:0] `w_data`

—— `r_data`

);

reg [Data.width-1:0] memory [0: 2^{Addr.width} - 1];

```
// write operation
```

```
always @ (posedge clk)
```

```
begin
```

```
    if (we)
```

```
        memory[w_addr] <= w_data;
```

```
end
```

```
// read operation
```

```
assign r_data = memory[r_address];
```

BRAM HDL Templates

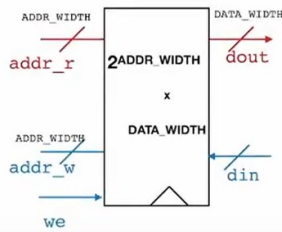
- Synchronous dual-port RAM $\begin{matrix} \nearrow P_1 \text{ } x, w \\ \searrow P_2 \text{ } r, w \end{matrix}$
- Simple Synchronous dual-port RAM $\begin{matrix} \nearrow P_1 \text{ } x \\ \searrow P_2 \text{ } w \end{matrix}$
- Synchronous single-port RAM $\rightarrow P \text{ } x, w$

this time
read is also synchronous

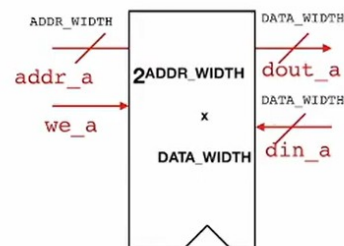
Synchronous dual-port RAM



"Simple" synchronous dual-port RAM



Synchronous single-port RAM



ROM HDL

* FPGA does not have any dedicated ROM

```
module rom_with_file (  
    input  [2:0] oddr,  
    output [1:0] data  
);
```


reg [1:0] rom [0:7]

initial
\$ readmemb ("truth.table.mem",
rom);

will not
be interpreted
by FPGA, only
considered
while simulation

read memory
binary

call system
function

b hex

assign data = rom [addr];

endmodule

↳ we can make sysc ROM

using synthesizer directive

(* rom_style = "block" *)

reg [1:0] rom [0:7];

